mmap 的数据结构：vmspace → vm_area_struct → 页表 → 物理页

① 进程用户虚拟地址空间（TTBR0，256 GiB）

保留 256 MiB（守护）

0x40_0000_0000

MMAP_TOP 0x3f_f000_0000

/dev/fb0 rw-s device
remap_pfn_range, Normal-NC

/dev/mem r--s device io
pgoff = 0x3f003 (系统定时器)

data.bin rw-s file
共享文件页，可回写

lib.dat rw-p file
MAP_PRIVATE：私有副本

MAP_ANON rw-s anon
shm_object 管理的共享匿名页

MAP_ANON rw-p anon
首次访问才分配的零页

↑ get_unmapped_area() 自顶向下分配

（空洞，未映射）

sbrk 不能越过最低的 VMA

MMAP_MIN 0x10_0000_0000

……

vm->sz

heap（sbrk / growproc）

stack + guard

text / data / bss
exec 装载，uvmcopy 复制

0

[0, sz) 仍由 exec/sbrk 管理，与原来一样；
mmap 区域全部在 MMAP_MIN 之上，由 VMA 描述。

② 内核数据结构（Linux mm_struct / vm_area_struct）

struct vmspace (mm_struct)
lock spinlock：页表项、mmap 链表
mmap_lock rw_semaphore
读：缺页、msync
写：mmap / munmap / exit
pagetable TTBR0 根
sz [0, sz) 的大小
mmap ——► 按地址排序的 VMA 链表
camera_mapped /dev/video0 映射计数

vm_area_struct (file, MAP_SHARED)
vm_start / vm_end 0x3f_ef..000 / +2 页
vm_flags VM_READ|VM_WRITE|VM_SHARED
vm_pgoff 文件偏移 / 4096
vm_file ► struct file ► inode（页缓存 key）
vm_page_prot PTE_NORMAL
vm_next ►

vm_area_struct (anon, MAP_SHARED)
vm_flags VM_READ|VM_WRITE|VM_SHARED
vm_file 0
vm_shmkey ► shm_object { ref = VMA 数 }
vm_next ►

vm_area_struct (device, /dev/fb0)
vm_flags ...|VM_SHARED|VM_IO|VM_PFNMAP
vm_page_prot PTE_NORMAL_NC（写合并）
vm_ops 驱动可选 open/close/fault
vm_file ► /dev/fb0（video0：VM_NOFILEREF）

struct file_operations
read_at / write_at 普通文件：通用缺页路径
mmap(file, vma) 设备：驱动自己装 PTE

改动：mmap 从 (f, addr, len, prot, flags, off)
改成 Linux 的 int (*mmap)(file, vma)，删掉 munmap，
munmap 由 VMA 统一处理。

③ PTE 指向的物理页，以及谁拥有它

私有页（anon / MAP_PRIVATE file）
kalloc() 一页，只属于这一个 PTE
缺页时：清零，或 read_at 读入文件内容（尾部补 0）
fork：dup_mmap 复制一份（本内核没有 COW）
munmap：清 PTE → flush_tlb → kfree

pcache_page（MAP_SHARED 文件）
key = inode 指针，index = 文件页号
mapcount = 映射它的 PTE 个数
同一 inode 的所有 mmap（跨进程、跨 open）共用一页
先装成只读 = 干净；第一次写 → 权限缺页 → 改成 RW = 脏
munmap / msync / exit：把可写 PTE 的页 write_at 回文件
最后一个 PTE 消失时释放（数据已在文件里）

shm_object + pcache_page（共享匿名）
key = shm_object 指针，ref = 共享它的 VMA 数
没有后备文件，所以页不能随 mapcount 归零而丢弃：
父进程映射、子进程先写的那一页也要留给父进程
最后一个 VMA 释放时（shm_put）才 kfree 所有页

设备帧（VM_PFNMAP，PTE_SPECIAL）
/dev/mem 0x3f003000 Device-nGnRnE 定时器寄存器
/dev/fb0 0x3c100000 Normal-NC GPU 帧缓冲
/dev/video0 0x07200000 Normal-NC Unicam DMA
没有拥有者：munmap 只清 PTE，fork 直接复制 PTE
copyin/copyout 只在内核直映覆盖时才经 P2V 访问

L3 PTE 中用到的位
AP[7:6] 01 = 用户 RW 11 = 用户 RO
共享文件页：RW 就表示“脏”（软件脏位）
UXN/PXN 无 PROT_EXEC 时置 UXN；用户页总置 PXN
AttrIdx vm_page_prot：NORMAL / NORMAL_NC / DEVICE
bit 56 PTE_SPECIAL：设备帧，不 kfree，不随意 P2V
bit 55 PTE_SHARED：原有的 libc.so 文本共享页
Cortex-A53 没有硬件脏位(DBM)，所以用权限缺页记脏

对应代码：include/kernel/mm.h，kernel/mm/mmap.c；页表位在 kernel/aarch64.h。
锁顺序（外 → 内）：vm->mmap_lock → inode 锁 → vm->lock → pcache.lock